

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

During a billing dispute resolution, your agent successfully retrieves customer info via `get_customer` and order details via `lookup_order`, but when attempting to call `process_refund`, the tool returns a timeout error. The agent has enough information to explain the charges and verify refund eligibility, but cannot actually process the refund due to the backend failure.

What approach best balances first-contact resolution with appropriate error handling?

- ☐ A. Escalate immediately to a human agent since the refund action cannot be completed -ab
- ☒ B. Explain the billing, confirm refund eligibility, acknowledge the system issue preventing immediate processing, and offer escalation or retry later -ab
- ☐ C. Implement automatic retries with exponential backoff for `process_refund`, keeping the conversation open until the refund is successfully processed -ab
- ☐ D. Confirm the refund will be processed and close the conversation, since the system has all necessary information to complete it automatically -ab

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

The agent verifies customer identity through a multi-step process before resetting passwords. During testing, you notice that after the customer answers the third verification question, the agent asks them to provide their name again, as if the earlier exchange never happened.

What's the most likely cause of this behavior?

- ☐ A. The prompt lacks instructions telling Claude to remember information across multiple exchanges. -ab
- ☐ B. The verification tool is clearing the agent's internal state after each successful validation step. -ab
- ☐ C. Claude's memory retention is limited to two conversational turns by default, requiring explicit configuration to extend it. -ab
- ☒ D. The conversation history isn't being passed in subsequent API requests. -ab

Question 2 of 60

☐ Mark for Comment ☐ Flag for Review

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your `process_refund` tool returns two types of errors: technical errors ("503 Service Unavailable", "Connection timeout") that are transient (~5% of calls), and business errors ("Order exceeds 30-day return window", "Item already refunded") that are permanent (~12% of calls). Monitoring shows the agent wastes 3-4 turns retrying business errors that can never succeed. Currently, both error types return only a plain text message to Claude.

What's the most effective way to reduce wasted retries while improving customer-facing response quality?

- ☐ A. Add a `check_refund_eligibility` tool that must be called before `process_refund` to prevent business rule violations. 
- ☒ B. Return structured error responses with `retriable: false` for business errors and a customer-friendly explanation for Claude to use. 
- ☐ C. Implement automatic retry logic at the tool level for technical errors only, passing business errors to Claude without retries. 
- ☐ D. Add few-shot examples showing how to distinguish retriable from non-retriable errors by parsing error message text. 

Question 4 of 60

☐ Mark for Comment ☐ Flag for Review

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

A customer raises three separate issues during one session: a refund inquiry (turns 1-15), a subscription question (turns 16-30), and a payment method update (turns 31-45). At turn 48, the customer asks "What happened with my refund?" The conversation is approaching context limits.

What strategy best maintains the agent's ability to address all issues throughout the session?

- ☐ A. Implement sliding window context that retains the most recent 30 turns. ab
- ☐ B. Summarize earlier turns into a narrative description, preserving full message history only for the active issue. ab
- ☒ C. Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer. ab
- ☐ D. Rely on MCP tools to re-fetch relevant information on demand when the customer references earlier issues. ab

Question 5 of 60

☐ Mark for Comment ☐ Flag for Review

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

After expanding the agent's MCP tools with delivery-specific capabilities (`check_delivery_status`, `contact_driver`, `issue_credit`, `apply_promo_code`, `update_delivery_address`, `reschedule_delivery`), the total tool count has grown from 4 to 10. Your evaluation suite shows tool selection accuracy has dropped from 88% to 71%. Log analysis reveals the majority of errors involve the agent selecting between semantically overlapping tools — calling `issue_credit` when `process_refund` was correct, and calling `check_delivery_status` when `lookup_order` already returns the needed data.

Which approach structurally eliminates the semantic overlap identified in the logs as the error source?

- ☒ A. Consolidate semantically overlapping tools — merge `issue_credit` and `process_refund` into a single `resolve_compensation` tool with an `action` parameter, and fold `check_delivery_status` into `lookup_order` with an optional `include_tracking` flag. 
- ☐ B. Split the tools across two sub-agents — a "financial resolution" agent with `process_refund`, `issue_credit`, and `apply_promo_code`, and a "delivery operations" agent with the remaining delivery tools — with a coordinator routing between them. 

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Compliance requires that refunds exceeding \$500 must automatically escalate to a human agent—this rule cannot be left to model discretion. Despite clear system prompt instructions, production logs show the agent occasionally processes high-value refunds directly (3% failure rate).

How should you achieve guaranteed compliance?

- ☐ A. Add few-shot examples to the prompt showing correct escalation behavior at various refund amounts (\$400, \$500, \$600). -ab
- ☐ B. Modify the refund tool to return an error with message "Amount exceeds policy limit—please escalate" when threshold is exceeded. -ab
- ☒ C. Implement a hook to intercept tool calls; when the refund process amount exceeds \$500, block it and invoke human escalation. -ab
- ☐ D. Strengthen the system prompt with emphatic language: "CRITICAL POLICY: Refunds over \$500 MUST trigger human escalation. NEVER process these directly." -ab

Question 7 of 60

☐ Mark for Comment

☐ Flag for Review

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

During testing, you find that when a customer says "I need a refund for my recent purchase," the agent calls `process_refund` immediately — but populates the required `order_id` parameter with a plausible-looking but fabricated value instead of first calling `lookup_order` to retrieve the actual order ID. The refund call fails because the fabricated ID doesn't exist.

Which change directly addresses the root cause of the agent fabricating the `order_id` value?

- ☐ A. Pre-parse incoming customer messages to extract any order IDs mentioned, and inject them into the conversation context before passing to Claude. -ab
- ☒ B. Update the `process_refund` tool description to explicitly state that `order_id` must be obtained from a prior `lookup_order` call and must never be assumed or invented. -ab
- ☐ C. Switch `tool_choice` from `"auto"` to `"any"` to force the agent to make a tool call on every turn. -ab
- ☐ D. Add server-side validation that checks whether the `order_id` exists in your database before executing the refund, returning an error to the agent if not found. -ab

Question 8 of 60

☐ Mark for Comment ☐ Flag for Review

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Production logs show that when the agent handles complex billing disputes requiring 6+ tool calls, it sometimes exhausts its ``max_turns`` limit after gathering data but before completing resolution or escalating. The team's goal is to guarantee that every customer interaction ends with either a completed resolution or a human handoff, regardless of how the agent loop terminates.

Which approach achieves this guarantee?

- ☐ A. Split the workflow into two sequential agent invocations — a first agent gathers information via ``get_customer`` and ``lookup_order``, then a second agent receives that data and handles ``process_refund`` or ``escalate_to_human``, each with separate turn budgets. 
- ☒ B. Add orchestration-layer code that checks the agent's outcome after each loop termination — if the loop ended without a completed resolution or escalation, programmatically call ``escalate_to_human`` with the accumulated conversation context and tool results. 
- ☐ C. Implement a pre-tool-use hook that counts tool invocations and terminates the loop with an ``escalate_to_human`` call if the count reaches a threshold. 

Question 9 of 60

☐ Mark for Comment

☐ Flag for Review

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

A customer returns 4 hours after their initial session about the same billing dispute. The previous 32-turn session contains `lookup_order` results showing "Status: PENDING, Expected resolution: 24-48 hours." In testing, you observe that when resuming sessions with stale tool results, the agent often references the outdated data in responses (e.g., "I see your refund is still being processed") even after subsequent fresh tool calls return different information.

What approach most reliably handles returning customers?

- ☒ A. Start a new session, inject a structured summary of the previous interaction (issue type, actions taken, resolution status), then make fresh tool calls before engaging. ab
- ☐ B. Resume with full history but filter out previous `tool_result` messages before resuming, keeping only the human/assistant turns so the agent must re-fetch needed data. ab
- ☐ C. Resume with full history and configure the agent to automatically re-call all previously-used tools at session start to ensure data freshness. ab
- ☐ D. Resume with full history and add a system prompt ab

Question 10 of 60

☐ Mark for Comment ☐ Flag for Review

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

A customer contacts the agent about a warranty claim on a power drill. Resolving this requires multiple sequential tool calls: `get_customer` to look up their account, `lookup_order` to find the purchase details, and then either `process_refund` or `escalate_to_human` depending on warranty eligibility. You're implementing the agentic loop that orchestrates these steps using the Claude API.

What is the primary mechanism your application uses to determine whether to continue the loop or stop?

- ☐ A. You check whether Claude's response contains a text content block — if text is present, the agent has produced its final answer and the loop should exit. -ab-
- ☒ B. You check the `stop_reason` field in each API response — the loop continues while it equals `"tool_use"` and exits when it changes to `"end_turn"` or another terminal value. -ab-
- ☐ C. You manually set the `tool_choice` parameter to `"none"` after the final expected tool call to force Claude to stop requesting tools. -ab-

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Production logs reveal inconsistent error handling: when `lookup_order` fails, the agent sometimes retries 5+ times (wasteful when the order ID doesn't exist), sometimes escalates immediately (premature for temporary network issues), and sometimes asks users for clarification (inappropriate when the issue is a backend permission error). Investigation shows your MCP tool returns uniform error responses: `{"isError": true, "content": [{"type": "text", "text": "Operation failed"}]}`. The agent cannot distinguish between error types.

What's the most effective improvement?

- ☒ A. Enhance error responses with structured metadata: include `errorCategory` (transient/validation/permission), `isRetryable` boolean, and a description of what caused the failure. ⌵
- ☐ B. Add few-shot examples to the system prompt demonstrating how to interpret error message patterns and select appropriate responses for each. ⌵
- ☐ C. Implement retry logic with exponential backoff in your MCP server for all errors, returning to the ⌵

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

When the agent calls `lookup_order` and receives order details showing the item was purchased 45 days ago, how does the agentic loop determine whether to call `process_refund` or `escalate_to_human` next?

- ☐ A. The agent follows a pre-configured decision tree mapping order attributes to specific tool calls. -ab
- ☐ B. The agent executes the remaining steps in a tool sequence planned at the start of the request. -ab
- ☐ C. The orchestration layer automatically routes to the next tool based on the order's status field. -ab
- ☒ D. The order details are added to the conversation and the model reasons about which action to take. -ab

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your agent is handling a billing dispute. After calling `get_customer` and `lookup_order`, it identifies that the dispute involves a promotional pricing error requiring manager approval—beyond the agent's authorization level.

How should the workflow handle this mid-process escalation?

- ☒ A. Compile a structured handoff with customer details, order info, and the identified issue before calling `escalate_to_human`. ab
- ☐ B. Persist the complete conversation and tool response history to a database, then call `escalate_to_human` with a reference ID. ab
- ☐ C. Call `escalate_to_human` passing only the customer's original message. ab
- ☐ D. Attempt the refund with `process_refund` anyway, escalating only if the system rejects the transaction. ab

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

You're implementing the escalation logic for when the agent should call `escalate_to_human`. Your team proposes four different approaches for triggering escalation.

Which approach will most reliably identify cases that genuinely require human intervention?

- ☐ A. Build a rules engine that maps specific issue types, customer segments, and product categories to escalation decisions, removing the need for model judgment calls. -ab
- ☐ B. Configure the agent to escalate after three consecutive tool calls that fail to resolve the customer's stated issue, ensuring a reasonable attempt before involving a human. -ab
- ☐ C. Implement sentiment analysis that monitors for frustration indicators (negative language, repeated questions, exclamation marks) and trigger escalation when the frustration score exceeds a configured threshold. -ab
- ☒ D. Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress. -ab



Question 15 of 60

☐ Mark for Comment ☐ Flag for Review

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Anthropic's tool use documentation states: "Write instructive error messages. Instead of generic errors like 'failed', include what went wrong and what Claude should try next." A billing dispute agent uses `lookup_order`, which catches all exceptions and returns a `tool_result` with `is_error: true` and the message "Tool execution failed". Monitoring shows two failure modes: the agent retries the identical call until hitting the turn limit, or it immediately calls `escalate_to_human` without trying alternative tools.

Which change follows the documented recommendation and gives Claude the information it needs to select the correct recovery action for each error type?

- ☐ A. Add an error classification step in the agentic loop that intercepts tool errors before Claude sees them, tags each as "retry," "try_alternative," or "escalate," and appends that recommendation to the tool result. 
- ☐ B. Implement retry logic with exponential backoff inside each tool implementation so transient errors are resolved transparently within the tool before any failure result is surfaced to Claude in the agentic loop. 

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

`escalate_to_human` without trying alternative tools.

Which change follows the documented recommendation and gives Claude the information it needs to select the correct recovery action for each error type?

- ☐ A. Add an error classification step in the agentic loop that intercepts tool errors before Claude sees them, tags each as "retry," "try_alternative," or "escalate," and appends that recommendation to the tool result. ⚑
- ☐ B. Implement retry logic with exponential backoff inside each tool implementation so transient errors are resolved transparently within the tool before any failure result is surfaced to Claude in the agentic loop. ⚑
- ☐ C. Remove ``is_error: true`` and return the error details as normal tool content, so Claude reasons about the response as data rather than treating it as a flagged failure condition that biases retry behavior. ⚑
- ☒ D. Return error-type-specific messages with ``is_error: true``, e.g., ``"Order not found—try get_customer to search by phone"`` for data errors and ``"Database timeout (transient)—retry should succeed"`` for infrastructure errors. ⚑

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're implementing a new payment processing module that must follow your project's established patterns for database transactions, error handling, and audit logging. You've identified three existing modules that exemplify these patterns: `db_utils.py`, `error_handlers.py`, and `audit_logger.py`. This is a one-off integration task—these patterns are well-documented in your team wiki and don't need additional project-level documentation.

What's the most effective approach?

- ☒ A. Use @ references to include the three modules directly in your prompt, giving Claude concrete code examples of the patterns to follow. -ab-
- ☐ B. Ask Claude to explore your codebase to find and understand the transaction, error handling, and logging patterns before generating the new module. -ab-
- ☐ C. Add documentation of each pattern to your CLAUDE.md file, establishing them as project conventions that Claude will apply automatically. -ab-
- ☐ D. Describe the patterns from the three modules in natural language in your prompt, explaining the transaction handling approach, error format, and -ab-

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're implementing a caching layer for API responses to speed up the `/products` endpoint. You have a rough idea—Redis with a 5-minute TTL—but you're new to production caching and aren't sure what other considerations a robust implementation requires.

What's the most effective way to start your iterative workflow?

- ☐ A. Use plan mode to analyze the current `/products` endpoint implementation, then provide your caching requirements once Claude explains how the existing code is structured. ab
- ☐ B. Start with a minimal request: "Add Redis caching to `/products` with 5-minute TTL." Add features and fix issues through follow-up prompts as problems surface during testing. ab
- ☐ C. Write a specification with your known requirements and "TBD" markers for uncertain areas, having Claude propose solutions for each TBD as it implements. ab
- ☒ D. Ask Claude to interview you about the caching requirements before implementing, surfacing considerations like invalidation strategies, cache layers, consistency guarantees, and failure modes. ab

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team's CLAUDE.md includes a rule: "Use 4-space indentation and always run Prettier formatting." Despite this, code reviews reveal that roughly 30% of files Claude Code generates use inconsistent formatting — sometimes 2-space indentation, sometimes missing trailing commas. Adding emphasis ("IMPORTANT: You MUST use Prettier formatting") reduces violations to about 15%, but doesn't eliminate them.

What is the most effective way to ensure all generated code is consistently formatted?

- ☒ A. Configure a PostToolUse hook with an Edit|Write matcher that automatically runs Prettier on each file Claude modifies. 
- ☐ B. Add a Stop hook with a prompt-based check that evaluates whether generated code follows formatting standards and prompts Claude to fix violations. 
- ☐ C. Split the formatting rules into path-scoped .claude/rules/ files that load when Claude works on matching file types. 
- ☐ D. Extract the formatting rules into a dedicated skill that Claude loads automatically when generating. 

Question 19 of 60

☐ Mark for Comment

☐ Flag for Review

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team has three requirements for Claude Code's behavior in your project: 1. Claude must never modify files in the ``db/migrations/`` directory 2. Claude should prefer your custom logging module over ``console.log`` 3. All TypeScript files must be auto-formatted with Prettier after every edit All three are currently written as instructions in your project's CLAUDE.md. During a complex refactoring session, a developer discovers that Claude edited a migration file, violating requirement #1.

How should you restructure these requirements across Claude Code's configuration mechanisms?

- ☒ A. Add ``Edit(..db/migrations/**)`` to ``permissions.deny`` in the project settings, keep the logging preference in CLAUDE.md, and add a ``PostToolUse`` hook on the Edit tool that runs Prettier on changed TypeScript files. 
- ☐ B. Configure hooks for all three: a ``PreToolUse`` hook script that blocks Edit calls targeting ``db/migrations/``, a ``PreToolUse`` hook script that adds logging convention context before edits, and a ``PostToolUse`` hook that runs Prettier after TypeScript edits. 
- ☐ C. Move all three requirements into ``..claude/rules/`` as path-scoped rules: one targeting ``db/migrations/**`` that forbids editing those files, and others targeting ``**/*.ts`` for the logging convention and formatting 

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team is configuring MCP servers in Claude Code. You want to add a shared venue lookup server that all team members should have access to, and you personally want to add an experimental music playlist server that only you are testing.

Which configuration approach correctly applies MCP server scopes?

- ☐ A. Add venue server to `~/.claude.json` and playlist server to `~/.mcp.json` 
- ☐ B. Add both servers to your local `~/.claude.json` 
- ☒ C. Add venue server to `~/.mcp.json` and playlist server to `~/.claude.json` 
- ☐ D. Add both servers to the project-level `~/.mcp.json` file 

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're implementing a complex graph traversal algorithm with specific performance requirements and edge cases to handle (disconnected nodes, cycles, weighted edges). You want to structure your workflow for efficient iterative refinement with Claude.

What approach will most effectively enable progressive improvement across multiple iterations?

- ☐ A. Provide Claude with a reference implementation from documentation, then ask it to rewrite the code to match your codebase style and add the required edge case handling, comparing outputs against the reference. -ab-
- ☐ B. Have Claude extensively research the algorithm and create a detailed implementation plan using extended thinking, then implement the complete solution based on that plan. -ab-
- ☒ C. Write a test suite covering expected behavior, edge cases, and performance requirements before implementation. Ask Claude to write code that passes the tests, then iterate by sharing test failures with each refinement request. -ab-

Question 22 of 60

☐ Mark for Comment ☐ Flag for Review

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team wants Claude to follow a detailed code review checklist (8 items covering API changes, test coverage, documentation, security, etc.) when reviewing pull requests. The team also uses Claude extensively for other tasks: writing new features, debugging production issues, and generating documentation. Currently, developers paste the checklist at the start of each review session.

Which approach best addresses this workflow need?

- ☐ A. Create a dedicated review subagent with the checklist embedded in its configuration. 
- ☐ B. Add the checklist to the project's CLAUDE.md file under a "Code Review" section. 
- ☒ C. Create a `/review` slash command containing the checklist, invoked when starting reviews. 
- ☐ D. Configure plan mode as the default for code review sessions. 

Question 23 of 60

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your infrastructure-as-code repository includes Terraform modules (`/terraform/`), Kubernetes manifests (`/kubernetes/`), and CI/CD pipeline scripts (`/pipelines/`). Each requires different conventions, but your single root CLAUDE.md has grown to 500+ lines. When developers work on Kubernetes files, Terraform-specific rules load into context unnecessarily, consuming tokens.

What is the best approach to reorganize so only relevant guidance loads when editing specific file types?

- ☐ A. Restructure the root CLAUDE.md into clearly labeled sections with headers (e.g., "## Terraform Conventions"), improving organization and readability. 
- ☐ B. Keep the root CLAUDE.md and use `@path/to/import` syntax to modularly include tool-specific guidance files from separate documents. 
- ☐ C. Create files in `.claude/rules/` with YAML frontmatter path-scoping (e.g., `paths: ["terraform/**/*"]`), loading rules only when editing matching files. 
- ☒ D. Split content into subdirectory CLAUDE.md files (`/terraform/CLAUDE.md`, `/kubernetes/CLAUDE.md`), so Claude loads directory-specific guidance. 

Question 24 of 60

☐ Mark for Comment

☐ Flag for Review

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

A security audit requires updating your authentication library from v2 to v3. The migration guide documents breaking changes: `authenticate()` now returns a Promise instead of accepting a callback, the `User` type has restructured fields, and three deprecated methods were removed. Grep shows the library is imported in 45 files across several modules.

What's the most effective approach?

- ☐ A. Create a custom slash command encapsulating the migration transformations, then execute it against each file without prior codebase exploration. -ab
- ☒ B. Enter plan mode to explore library usage across modules, map affected code paths, then create a migration strategy before implementing. -ab
- ☐ C. Paste the migration guide's breaking changes into your prompt and use direct execution to update all usages across the 45 files. -ab
- ☐ D. Update the dependency version, run the test suite, -ab

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You need to add a date validation check ensuring event dates are in the future. This requires adding a conditional statement to one existing function in a single file.

What is the most appropriate approach?

- ☒ A. Use direct execution to make the change -ab
- ☐ B. Enter plan mode first to create a detailed implementation strategy before making the change -ab
- ☐ C. Start with extended thinking mode enabled to ensure thorough reasoning about the validation logic -ab
- ☐ D. Enter plan mode to analyze how the validation might impact other parts of the reservation flow -ab

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You've asked Claude Code to build a PDF report generation feature. The initial implementation queries the database correctly, but the output has formatting issues: table columns are too narrow causing content truncation, dates display without proper formatting, and page break handling is incorrect. You've noticed these issues interact—changing column widths affects how dates render, and page breaks depend on content height.

What's the most effective approach for iterating toward a working solution?

- ☐ A. Show Claude an example of a correctly formatted report and ask it to match that output, rather than listing the specific technical issues. ab
- ☒ B. Address the column width issue first with specific measurements, verify it works, then fix date formatting within the corrected columns, then adjust page breaks—testing after each change. ab
- ☐ C. Provide all three issues in a single detailed message with exact specifications for each, allowing Claude to address them together in one update. ab

Question 27 of 60

☐ Mark for Comment

☐ Flag for Review

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team has connected a custom MCP server that provides DevOps workflow templates. The server exposes several MCP prompts (such as `deploy_checklist` and `incident_response`) in addition to tools.

How do these MCP prompts become accessible within Claude Code?

- ☐ A. They are automatically prepended to every conversation as additional system-level context, influencing Claude's behavior throughout the session. ⚑
- ☐ B. They are surfaced as `@`-mentionable resources alongside files, fetched and attached to your message when referenced. ⚑
- ☐ C. They are added to Claude Code's tool registry alongside the server's tools, invoked automatically by the model when relevant to the task. ⚑
- ☒ D. They appear as slash commands (e.g., `/mcp__servername__deploy_checklist`) that you can invoke, with arguments passed after the command name. ⚑

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're tasked with adding real-time updates to the application. This could be implemented using WebSockets, Server-Sent Events, or polling, each with different complexity, browser support, and infrastructure requirements.

What's the most effective way to begin this task?

- ☐ A. Use direct execution with a prompt asking Claude to analyze all approaches and implement the one it determines is best. -ab
- ☐ B. Use direct execution to implement polling first, then evaluate whether to upgrade to WebSockets later. -ab
- ☐ C. Start direct execution with WebSockets, then refactor if infrastructure issues arise. -ab
- ☒ D. Enter plan mode to explore the architecture, evaluate trade-offs, and present options for team approval before implementing. -ab

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You've asked Claude to write a data migration script, but the initial output doesn't correctly handle records with null values in required fields.

What's the most effective way to iterate toward a working solution?

- ☒ A. Provide a test case with example input containing null values and the expected output, then ask Claude to fix it. -ab
- ☐ B. Add "think harder about edge cases" to your prompt and request a complete rewrite of the migration logic. -ab
- ☐ C. Describe the null value problem in detail and ask Claude to regenerate the entire script with improved edge case handling. -ab
- ☐ D. Manually edit the generated code to fix the null handling, then continue working with Claude on other parts. -ab

Question 30 of 60

☐ Mark for Comment

☐ Flag for Review

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team frequently migrates React components to Vue. You've written a step-by-step workflow for Claude Code to follow during each migration, and you want every developer on the team to invoke it by typing `/migrate-component`. The workflow should stay in sync as the team iterates on it.

Where should you place the skill file?

- ☐ A. In the project's `.claude/settings.json` using a `skillOverrides` entry to register and define the workflow ⌵
- ☒ B. In `.claude/skills/migrate-component/SKILL.md` at the project root, committed to version control ⌵
- ☐ C. In `~/claude/skills/migrate-component/SKILL.md` on each developer's machine ⌵
- ☐ D. As a detailed instruction block in the project's root `CLAUDE.md` file ⌵

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer asks your agent to add comprehensive tests to a legacy codebase with 200 files and minimal existing test coverage. The engineer hasn't specified which modules to prioritize.

How should the agent decompose this open-ended task?

- ☒ A. Use Glob and Grep to map codebase structure, identify heavily-coupled modules, create a prioritized plan for high-impact areas, and revise as dependencies are discovered. -ab
- ☐ B. Start writing tests for the first module alphabetically, using test failures and imports to discover related files organically. -ab
- ☐ C. Systematically read all 200 files to create a complete function inventory before writing any tests, ensuring the testing plan accounts for every function before beginning. -ab
- ☐ D. Create a fixed testing schedule upfront based on directory structure, allocating equal effort to each top-level directory regardless of code complexity or business importance. -ab

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

appear in the `tools/list` response. However, you observe that the agent consistently uses `Grep` to search for import statements instead of calling `analyze_dependencies`—even when users explicitly ask about "code dependencies." Examining tool definitions reveals:

- MCP: `analyze_dependencies` — "Analyzes dependency graph"
- Built-in: `Grep` — "Search file contents for a pattern using regular expressions. Returns matching lines with line numbers and surrounding context."

What's the most effective approach to improve the agent's selection of MCP tools?

- ☐ A. Split `analyze_dependencies` into granular tools (`list_imports`, `resolve_transitive_deps`, `detect_circular_deps`) so each has a focused purpose less likely to overlap with `Grep`.
- ☐ B. Add routing instructions to the system prompt specifying that dependency-related questions should use MCP tools rather than `Grep`.
- ☐ C. Remove `Grep` from available tools when the MCP server is connected to eliminate functional overlap.
- ☒ D. Expand MCP tool descriptions to detail capabilities and outputs—e.g., "Builds dependency graph showing direct imports, transitive dependencies, and cycles."

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

1.5An engineer asks the agent to understand how the caching layer works before adding a new cache invalidation trigger. After initial Grep searches, the agent has identified that caching logic spans 15 files including decorators, middleware, and service classes (~8,000 lines total).

What's the most effective next step for building understanding while managing context constraints?

- ☐ A. Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation. 
- ☒ B. Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations. 
- ☐ C. Use Grep to search for "invalidate" and "expire" patterns across all files, then Read only those specific line ranges with minimal surrounding context. 
- ☐ D. Use Glob to find files matching common caching patterns (*cache*.py, caching/), prioritize the largest files by reading them first, then check smaller files for gaps. 

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent has spent 25 minutes exploring a game engine's rendering subsystem—reading shader code, buffer management, and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference "typical rendering patterns" rather than the specific ``VulkanPipeline`` and ``FrameGraph`` classes it discovered earlier.

What's the most effective approach?

- ☐ A. Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation. -ab
- ☐ B. Continue in the current context with more targeted prompts referencing the specific classes by name. -ab
- ☐ C. Use ``/clear`` to reset context completely, then start fresh with physics exploration using file paths from the project's `CLAUDE.md`. -ab
- ☒ D. Summarize key rendering findings, then spawn a sub-agent for physics exploration with that summary in its initial context. -ab

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer asks your agent to identify untested code paths in a legacy payment processing module spanning 45 files. After reading the first 8 source files, the agent's responses are becoming noticeably less accurate—it's forgetting previously discussed code patterns and hasn't yet located all test files or traced critical payment flows.

What's the most effective approach to complete this investigation?

- ☐ A. Clear context with `/clear`, then selectively re-read only the most critical files discovered so far, writing key findings to a scratchpad file that persists between context resets. ⚠
- ☐ B. Switch to using Grep to search for specific function names instead of reading full files, reducing the content loaded into context for remaining exploration. ⚠
- ☐ C. Document all current findings in a summary report, clear context completely, then use that report as the sole reference for continuing the investigation. ⚠
- ☒ D. Spawn subagents to investigate specific questions (e.g., "find all test files for payment processing," "trace refund flow dependencies") while the main agent coordinates findings and preserves high-level understanding. ⚠

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent has analyzed a complex service module—reading 23 source files, tracing request flows, and identifying error handling patterns. A developer wants to compare two testing strategies before committing to one: end-to-end tests with mocked external services vs. snapshot tests capturing expected outputs. They need to independently develop both approaches to evaluate trade-offs.

How should you manage the sessions?

- ☐ A. Export the analysis session's key findings to a file, then create two new sessions that reference this file. ⌕
- ☒ B. Resume the analysis session with `fork_session` enabled, creating a separate branch for each testing strategy. ⌕
- ☐ C. Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially. ⌕
- ☐ D. Start two fresh sessions, having each re-read the relevant source files before beginning. ⌕

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer used the agent yesterday to analyze a legacy authentication module, identifying two distinct refactoring approaches: extracting a microservice versus refactoring in-place. Today, they want to explore both approaches in depth—having the agent propose specific code changes for each—before deciding which to implement.

What's the most effective way to structure this exploration?

- ☒ A. Use ``fork_session`` to create two branches from yesterday's analysis, exploring one approach in each fork. -ab
- ☐ B. Resume yesterday's session to explore the first approach, then start a new session for the second, manually recreating the original context. -ab
- ☐ C. Start two fresh sessions, manually providing a summary of yesterday's analysis findings to establish context. -ab
- ☐ D. Resume yesterday's session and explore both approaches sequentially within the same conversation thread. -ab

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer's exploration subagent spent 30 minutes analyzing a legacy payment system, reading 47 files and documenting data flows. The session was interrupted when the engineer's connection dropped. While away, a teammate merged a PR that renamed two utility functions. The engineer wants to continue the same exploration.

What's the most effective approach?

- ☐ A. Launch a fresh subagent with a summary of prior findings. -ab
- ☐ B. Resume the subagent from its previous transcript without mentioning the changes—the architecture understanding remains valid. -ab
- ☒ C. Resume the subagent from its previous transcript and inform it about the renamed functions. -ab
- ☐ D. Launch a fresh subagent and include the prior transcript in the initial prompt for context. -ab

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your codebase exploration tool stores session IDs to allow engineers to continue investigations across work sessions. An engineer spent an hour yesterday analyzing a legacy authentication module, building context about its architecture and dependencies. They want to continue today. The session ID is valid, but version control shows 3 of the 12 files the agent previously read were modified overnight by a teammate's merge.

What approach best balances efficiency and accuracy?

- ☐ A. Resume the session and immediately have the agent re-read all 12 previously analyzed files -ab
- ☐ B. Start a fresh session to ensure the agent works with current codebase state without stale assumptions -ab
- ☐ C. Resume the session without informing the agent about the changed files -ab
- ☒ D. Resume the session and inform the agent which specific files changed for targeted re-analysis -ab

Question 40 of 60

☐ Mark for Comment ☐ Flag for Review

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer asks the agent to find all callers of a function before removing it. The function is defined in a core library but is also exposed through wrapper modules that rename the function for domain-specific use (e.g., `calculateTax` in the library becomes `computeOrderTax` in the orders module).

What exploration strategy will most reliably identify all callers?

- ☒ A. Read the library and wrapper modules to identify all exposed names for the function, then Grep for each name across the codebase. ⚑
- ☐ B. Use Grep to find all files that import from the library or wrapper modules, then read each file to check whether it uses the function. ⚑
- ☐ C. Search for the function name in project documentation to understand intended usage patterns and navigate to documented integration points. ⚑
- ☐ D. Use Grep to search for the function's original name ⚑

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer used Claude Code yesterday to investigate authentication flows in a legacy monolith, building up significant context over a 2-hour session. Today she wants to continue that specific investigation. She's worked on three other codebases since then and knows the session was named "auth-deep-dive".

How should she resume?

- ☐ A. Use `--session-id` with the UUID from yesterday's session transcript file -ab
- ☐ B. Start fresh and re-read the same files -ab
- ☒ C. Use `--resume auth-deep-dive` to load that specific session by name -ab
- ☐ D. Use `--continue` to pick up where the most recent conversation left off -ab

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

You've configured your Claude agent with three MCP servers: one for git operations, one for Jira ticket management, and one for documentation search.

When a user asks the agent to "create a branch for JIRA-123 and add documentation links to the ticket," how does the agent access tools across these servers?

- ☐ A. You must specify which MCP server to use for each turn, and the agent can only access one server's tools at a time. -ab
- ☒ B. Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent. -ab
- ☐ C. The agent automatically selects the most relevant server based on the request and loads only that server's tools. -ab
- ☐ D. The agent queries each server sequentially to determine which handles each tool, routing calls based on tool name prefixes. -ab

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your code review assistant needs to analyze pull requests and provide feedback on three aspects: code style compliance, potential security issues, and documentation completeness. Each aspect requires reading files, running analysis tools, and generating a report section. The review process follows the same three-step workflow for every PR.

Which task decomposition pattern is most appropriate for this workflow?

- ☐ A. Single comprehensive prompt—include all instructions in one prompt and let the model handle all three aspects simultaneously. ab
- ☐ B. Orchestrator-workers—have a central LLM analyze each PR to dynamically determine which checks are needed, then delegate to specialized worker LLMs for each identified subtask. ab
- ☒ C. Prompt chaining—break the review into sequential steps where each aspect (style, security, documentation) is analyzed separately, with outputs combined in a final synthesis step. ab
- ☐ D. Routing—classify each PR by type (feature, bugfix, refactor) first, then route to different review prompts optimized for that category. ab

Question 44 of 60

☐ Mark for Comment☐ Flag for Review

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer submits two requests: - Request A: "Rename the `getUserData` function to `fetchUserProfile` everywhere it's used." - Request B: "Improve error handling throughout the data processing module—add try/catch blocks, meaningful error messages, and ensure failures don't silently corrupt data."

For which request does specifying an explicit multi-phase workflow (such as analyze → propose → implement with review) most improve outcome quality?

- ☐ A. Both requests benefit equally 
- ☐ B. Neither request benefits significantly 
- ☒ C. Request B, the error handling task 
- ☐ D. Request A, the function rename task 

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

A developer asks the agent to investigate why a specific API endpoint intermittently returns 500 errors. The codebase has 200+ files and the developer doesn't know which components are involved. The agent must trace the error through routing, middleware, business logic, and database layers.

What task decomposition approach would be most effective?

- ☒ A. Have the agent dynamically generate investigation subtasks based on what it discovers at each step, adapting its exploration plan as new information about the error path emerges. -ab
- ☐ B. Have the agent first create a comprehensive plan mapping all code paths through the endpoint before beginning any file exploration or code reading. -ab
- ☐ C. Define a fixed sequence of investigation steps upfront—grep for error patterns, then read error handlers, then check database queries, then examine middleware—executing each step regardless of intermediate findings. -ab
- ☐ D. Run parallel worker agents that simultaneously investigate all four layers, then synthesize their findings to identify where the error originates. -ab

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Monitoring shows 12% of extractions fail Pydantic validation with specific errors like "expected float for quantity, got '2 to 3'". Retrying these requests without modification produces identical failures.

What's the most effective approach to recover from these validation failures?

- ☐ A. Set temperature to 0 to eliminate output variability and ensure consistent formatting ⌂
- ☒ B. Send a follow-up request including the validation error, asking the model to correct its output ⌂
- ☐ C. Pre-process source documents to standardize problematic formats before sending them for extraction ⌂
- ☐ D. Implement a secondary pipeline using a larger model tier to reprocess documents that fail validation ⌂



Question 47 of 60

☐ Mark for Comment☐ Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your pipeline uses a tool called `extract_metadata` with a JSON schema for paper details. You've also defined `lookup_citations` and `verify_doi` tools for enrichment. During testing, you notice that when users include requests like "extract the metadata and tell me how cited it is," Claude sometimes calls `lookup_citations` first, which fails because it needs the DOI that `extract_metadata` would provide.

What's the most effective way to ensure structured metadata extraction happens first?

- ☐ A. Set `tool_choice` to `"auto"` and reorder the tool definitions so `extract_metadata` appears first in the tools array, since Claude prioritizes earlier-listed tools. 
- ☒ B. Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` and process the enrichment requests in subsequent turns after receiving the extracted metadata. 
- ☐ C. Set `tool_choice` to `"any"` so Claude must use a tool, combined with system prompt instructions prioritizing `extract_metadata`. 

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system has been operating with 100% human review for 3 months. Analysis shows that extractions with model confidence $\geq 90\%$ have 97% accuracy overall. To reduce reviewer workload, you plan to automate high-confidence extractions.

Before deploying, what validation step is most critical?

- ☐ A. Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors. ab
- ☐ B. Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data. ab
- ☒ C. Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate. ab
- ☐ D. Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports. ab



You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline processes restaurant menus and must output structured JSON with fields for item names, descriptions, prices, and dietary tags. Some menus use inconsistent formatting—prices as "\$12" vs "12.00", dietary info as icons vs text.

What's the most reliable approach?

- ☐ A. Define a strict output schema and include format normalization rules in your prompt. 
- ☐ B. Use separate extraction calls for each field to ensure consistent handling of each type. 
- ☒ C. Extract data as-is and normalize formats in post-processing code after Claude returns. 
- ☐ D. Request multiple extraction attempts per document and select the most common format. 

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system extracts event metadata (date, location, organizer, attendee_count) from news articles using a JSON schema with all nullable fields. During evaluation, you observe the model frequently generates plausible but incorrect values for fields not mentioned in the article—for example, outputting "500" for attendee_count when the source contains no attendance information.

What's the most effective way to reduce these false extractions?

- ☐ A. Upgrade to a more capable model tier with improved instruction-following to reduce hallucination tendencies. -ab
- ☐ B. Add a post-processing step using a second LLM call to verify each extracted value exists in the source document. -ab
- ☒ C. Add prompt instructions to return null for any field where information is not directly stated in the source. -ab
- ☐ D. Make all schema fields required (non-nullable) with strict validation rules to ensure the model only outputs verifiable data. -ab

Question 51 of 60

☐ Mark for Comment

☐ Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

The system needs to extract candidate information (name, contact details, skills, work experience, education) from uploaded resumes. The extracted data must strictly conform to a predefined JSON schema, as missing required fields or incorrect data types will cause downstream validation failures.

What is the most reliable approach to ensure Claude's output consistently matches the schema?

- ☐ A. Parse Claude's text response with regex patterns to extract JSON objects, using retry logic for malformed responses. -ab
- ☒ B. Define a tool with an input schema matching your required JSON structure and extract the data from Claude's tool_use response. -ab
- ☐ C. Include detailed JSON formatting instructions and a template example in the system prompt, asking Claude to output only valid JSON. -ab
- ☐ D. Make two separate API calls—first extracting information as text, then asking Claude to format that text as JSON. -ab

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline validates outputs against JSON schemas, but you need to implement human review given limited reviewer capacity (they can handle approximately 5% of total extraction volume).

What's the most effective basis for selecting which extractions to route for human review?

- ☐ A. Route extractions for review only when downstream systems report data quality issues or processing failures. -ab-
- ☒ B. Route extractions where the model indicates low confidence or where source documents contain ambiguous or contradictory information. -ab-
- ☐ C. Randomly sample 5% of extractions for review. -ab-
- ☐ D. Route extractions containing specific high-priority entity types (e.g., financial figures, dates) for human review, regardless of extraction confidence. -ab-

Question 53 of 60

☐ Mark for Comment☐ Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system must extract event details from calendar invitations and output JSON that strictly conforms to a schema with fields for title, date, time, location, and attendees. Downstream systems reject any malformed or non-conformant JSON.

What approach provides the most reliable schema compliance?

- ☒ A. Define a tool with your target schema as input parameters and have Claude call it with the extracted data. 
- ☐ B. Include detailed JSON formatting instructions and the target schema in your prompt, then parse Claude's text response as JSON. 
- ☐ C. Pre-fill Claude's response with an opening brace to force JSON output, then complete and parse the response. 
- ☐ D. Append instructions like "Output only valid JSON matching the schema exactly" and implement retry logic to re-prompt when JSON parsing fails. 

Question 54 of 60



Mark for Comment



Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Testing reveals that when source documents are missing certain specifications, the model fabricates plausible-sounding values to satisfy your schema's required fields. For example, a document mentioning only dimensions receives a fabricated "weight: 2.3 kg" in the extraction output.

What schema design change most effectively addresses this hallucination behavior?

- ☒ A. Change fields that may not exist in source documents from required to optional, allowing the model to omit them 
- ☐ B. Add a "confidence" field alongside each specification where the model self-reports its certainty, then filter out low-confidence extractions 
- ☐ C. Implement semantic validation that verifies each extracted value appears in or can be inferred from the source document text 
- ☐ D. Add explicit instructions to the prompt stating "only extract information explicitly stated in the document; use placeholder text for missing values" 



Question 55 of 60



Mark for Comment



Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline processes invoices and extracts line items, subtotals, tax amounts, and grand totals. During evaluation, you discover that in 18% of extractions, the sum of extracted line item amounts doesn't match the extracted grand total—sometimes due to OCR errors in the source document, sometimes due to extraction mistakes by the model. Downstream accounting systems reject records with mismatched totals.

What's the most effective approach to improve extraction reliability?

- ☒ A. Add a "calculated_total" field where the model sums extracted line items alongside a "stated_total" field. Flag records for human review when values differ. -ab-
- ☐ B. Extract line items and totals independently, then use a separate validation model to reconcile discrepancies by determining which extracted values are most likely correct. -ab-
- ☐ C. Implement post-processing that automatically adjusts line item amounts proportionally when their sum doesn't match the stated total. -ab-
- ☐ D. Add few-shot examples demonstrating invoices where extracted line items sum correctly to the stated total, encouraging the model to produce mathematically consistent extractions. -ab-

Question 56 of 60

☐ Mark for Comment ☐ Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your schema includes a `skills: string[]` field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract all skills mentioned."

What's the most effective improvement?

- ☐ A. Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries. 
- ☒ B. Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills." 
- ☐ C. Enrich the schema to `{skill: string, confidence: float, source_quote: string}[]` to capture extraction metadata. 
- ☐ D. Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity. 

 Question 57 of 60☐ Mark for Comment ☐ Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline processes contracts that frequently include amendments. When a contract contains both original terms and later amendments (e.g., original clause specifies "30-day payment terms" while Amendment 1 changes this to "45 days"), the model inconsistently extracts one value or the other with no indication of which applies.

What's the most effective approach to improve extraction accuracy for documents with amendments?

- ☐ A. Implement post-extraction validation using pattern matching to detect amendments and flag those extractions for manual review. -ab-
- ☐ B. Add prompt instructions to always extract the most recent amendment value and ignore superseded original terms. -ab-
- ☐ C. Preprocess documents with a classifier that identifies and removes superseded sections before the main extraction step. -ab-
- ☒ D. Redesign the schema so amended fields capture multiple values, each with source location and effective date. -ab-

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

The system routes documents with extraction confidence below 85% to human review. A quarterly audit reveals that 12% of high-confidence extractions ($\geq 85\%$) also contain errors—cases where the model finds plausible-but-incorrect values. Error sources vary: comparison tables showing competitor specs, appendices referencing different product variants, and ambiguous phrasing the model misinterprets. You need a sustainable strategy to catch these high-confidence errors and measure whether improvements reduce the error rate over time.

What approach is most effective?

- ☐ A. Lower the confidence threshold from 85% to 70%, routing a larger volume of extractions to human review. ab
- ☒ B. Implement stratified random sampling reviewing a fixed percentage of high-confidence extractions weekly, enabling error rate measurement and novel pattern detection. ab
- ☐ C. Add a verification pass that re-extracts from each high-confidence document, flagging cases where the two extraction attempts produce different results. ab
- ☐ D. Implement heuristic rules that flag documents containing comparison tables or appendices for review regardless of confidence score. ab

Question 59 of 60

☐ Mark for Comment

☐ Flag for Review

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system has been running for 3 weeks and human reviewers have corrected 847 extractions. Analysis reveals a recurring pattern: when recipes use informal measurements like "a handful" or "a splash," the model either invents specific amounts or leaves fields empty—accounting for 23% of all corrections.

How should you use this feedback to improve extraction accuracy?

- ☐ A. Implement a post-processing layer that uses pattern matching to detect informal measurement phrases in source text and automatically populate values when the extraction is empty. ab
- ☐ B. Fine-tune the model on the 847 corrected extractions. ab
- ☒ C. Add few-shot examples to your prompt demonstrating correct handling of informal measurements—extracting them verbatim rather than converting or omitting them. ab
- ☐ D. Update your JSON schema to add a "measurement_type" enum field (precise/informal). ab

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction system implements automatic retries when validation fails. On each retry, the specific validation error is appended to the prompt. This retry-with-error-feedback approach resolves most failures within 2-3 attempts.

For which failure pattern would additional retries be LEAST effective?

- ☐ A. The model extracts citation counts as locale-formatted strings ("1,234") when the schema requires integers -ab
- ☐ B. The model extracts dates as ISO 8601 datetime strings ("2023-03-15T00:00:00Z") when the schema requires only the date portion (YYYY-MM-DD) -ab
- ☐ C. The model extracts keywords as a nested object organized by category when the schema requires a flat array of strings -ab
- ☒ D. The model extracts "et al." for co-authors when the full list exists only in an external document not in the input -ab

TEST OBJECTIVES

PERCENT CORRECT

Design orchestration-layer safeguards ensuring every agent session ends with a completed resolution or human escalation, regardless of how the agentic loop terminates.

100%

Design structured handoff packages that preserve accumulated context, findings, and authorization state when transferring control between agent steps or to a human operator.

100%

Apply session resumption techniques—including targeted re-analysis of changed files and context injection—to restore agent state accurately without repeating prior work.

100%

Decompose complex tasks into dynamically generated subtasks that adapt as new information is discovered, rather than executing a fixed sequence regardless of intermediate findings.

100%

Apply PreToolUse and PostToolUse hook patterns to enforce business rules and policy constraints not delegated to model discretion.

100%

Explain how the agentic loop uses model responses and stop_reason signals to decide whether to continue tool execution or terminate and return a final response.

100%

Create and deploy custom slash commands in the correct project or user directory so they are available to intended users and invoked on demand for task-specific workflows.

100%

Structure iterative refinement workflows by providing concrete input-output examples, targeted feedback on specific failures, and batched issue descriptions for consolidated

75%